

Software Design Description

Francanglais Compiler and Dataset Platform

Architecture and Explained UML Workflows

Group members and matricules:

Name 1 — Matricule 1

Name 2 — Matricule 2

Name 3 — Matricule 3

Version 1.1 — September 19, 2026

Group submission. Complete the names and matricules before submission.

Design status: the collector workflows are source-grounded. Compiler/service elements are explicitly proposed. Diagrams do not certify requirements or planned sprints as completed.

Contents

1	Design Basis and UML Reading Guide	4
1.1	Evidence and scope	4
1.2	Notation and abstraction	4
2	Architecture and Deployment	4
2.1	Component dependencies	4
2.2	Deployment topology	5
3	Static Structure and Data Design	6
3.1	Application classes and supporting modules	6
3.2	Persistence and invariants	9
4	User Goals and Control Flow	9
4.1	Use-case diagram	9
4.2	Entry-collection activity diagram	10
4.3	Entry workflow state diagram	11
5	Sequence Diagrams and Interaction Explanations	12
5.1	Common sequence conventions	12
5.2	Collect and save with optional attachment	12
5.3	Search and select an entry	14
5.4	Edit an existing entry	15
5.5	Confirm or cancel deletion	16
5.6	Record and stage microphone audio	17
5.7	Play audio and fall back to the operating system	19
5.8	Refresh statistics	20
6	Interfaces, Errors and Design Limitations	21
6.1	Failure boundaries	22
6.2	Privacy and operational boundaries	22
7	Verification, Traceability and Change Impact	22
8	Proposed Evolution	23

List of Figures

1	Component dependencies and the proposed analysis boundary	5
2	Single-workstation deployment and optional future analysis host	6
3	Application classes, module dependencies and cardinalities	8
4	Collector actors, mandatory sub-behavior and optional audio extensions . . .	10
5	Collection activity with optional media and save-time validation	11
6	Conceptual draft, selection, edit and deletion states	12
7	Save interaction with optional attachment and validation branches	14
8	Searching CSV entries and selecting a cached result	15
9	Selected-entry validation and read-modify-write editing	16
10	Selection guard, confirmation and conditional deletion	17
11	Recorder start, stream lifetime, stop and staged WAV creation	19
12	Playback existence check, library attempt and OS fallback	20
13	Statistics refresh, repeated grouping and safe empty-data rendering	21

1 Design Basis and UML Reading Guide

1.1 Evidence and scope

The verified source consists of `data_collector/App.py`, `data_collector/dataset.py` and `data_collector/audio_utils.py`. These implement a local tabbed CustomTkinter application, CSV helpers and optional audio support. The compiled frontend bundle and cached Python bytecode do not provide sufficient source evidence for current backend routes or compiler behavior.

This revision explains all thirteen diagrams, including seven sequence diagrams. The SRS defines FR/NFR identifiers, sixteen user stories and four proposed two-week sprints. References here link design decisions to those requirements rather than inventing completed sprint history.

1.2 Notation and abstraction

- A **class** rectangle lists selected attributes and operations. The hollow triangle points to a superclass; a filled diamond sits at the whole end of composition. Multiplicities apply to associations, not dependencies.
- **Stereotypes** distinguish external library classes, function modules, logical storage and dictionary records from real application classes.
- In **sequence views**, solid synchronous messages start receiver activations; dashed replies finish them. Nested self-calls have nested boxes. Actor activations span user interactions, not processor execution.
- **alt**, **opt** and **loop** fragments mean mutually exclusive alternatives, conditional behavior and repetition respectively. Notes describe internal computation or deliberately omitted low-level calls.
- In **activity/state views**, guards in brackets select paths. An activity final node ends the illustrated attempt, not the desktop process.
- Amber proposed architecture areas are not deployed baseline components. Dashed component dependencies do not describe message timing.

Module and file-I/O lifelines are functional abstractions. Repeated storage initialization, widget calls and `load_all` calls can be condensed when their expansion would hide the scenario's purpose; this is stated beside each view. Shown communications retain explicit, balanced activations.

2 Architecture and Deployment

2.1 Component dependencies

Trace: FR-01–FR-14; proposed FR-15/FR-16; US-01–US-14.

Source evidence: imports and calls in `App.py`; persistence/audio helper modules.

Figure 1 answers *which module depends on which responsibility?* Presentation owns form state and user commands. Dataset functions own row initialization, reading, appending,

rewriting and counts. Audio utilities own recording, copying and playback. CSV and audio artifacts are separate stores, connected by a filename rather than a database foreign key.

Read the dashed arrows from caller/dependent to supplier: the presentation calls both helper modules; the helpers access their respective local files. These are ordinary Python calls, not HTTP services. The proposed adapter and language-analysis pipeline are separate so a missing compiler does not prevent collection. Their read-only access is a design target, not a current API.

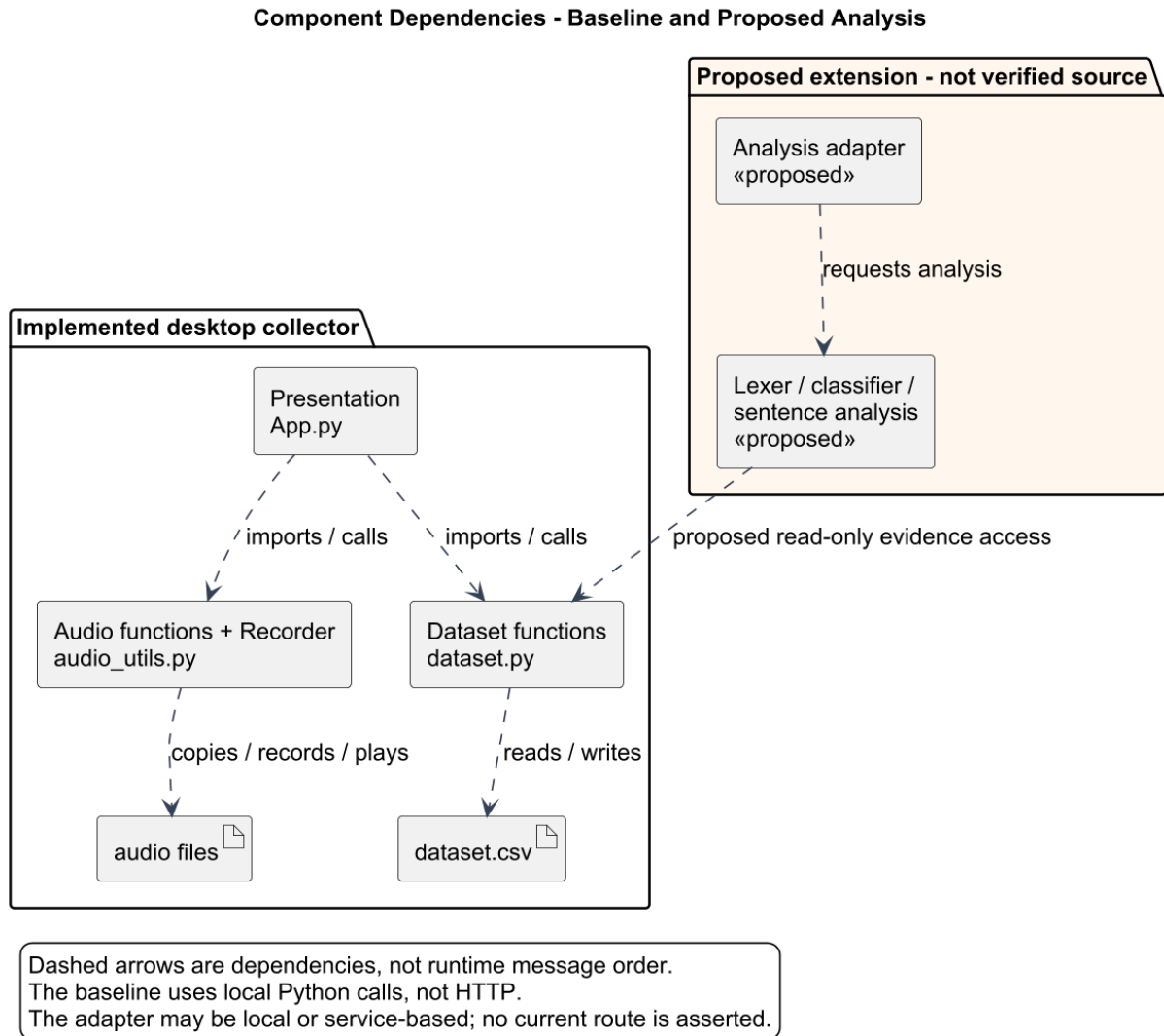


Figure 1: Component dependencies and the proposed analysis boundary

2.2 Deployment topology

Trace: NFR-04, NFR-05, NFR-08, NFR-12; proposed FR-15/FR-16.

Source evidence: module-relative paths and platform audio branches.

Figure 2 answers *where do code and data execute or reside?* A contributor workstation contains the Python/Tcl-Tk environment, source artifacts, CSV, media and OS audio services. A microphone is a device, not a dataset actor. Solid node connections identify device/communication paths; dashed artifact dependencies show imports and file access.

The optional analysis host is explicitly future deployment. It would need an approved corpus copy and an agreed transfer process; it cannot read a user's local CSV merely because

an arrow is drawn across machines. HTTP is only an option if a web adapter is selected. No hosting, synchronization, authentication or currently available endpoint is implied.

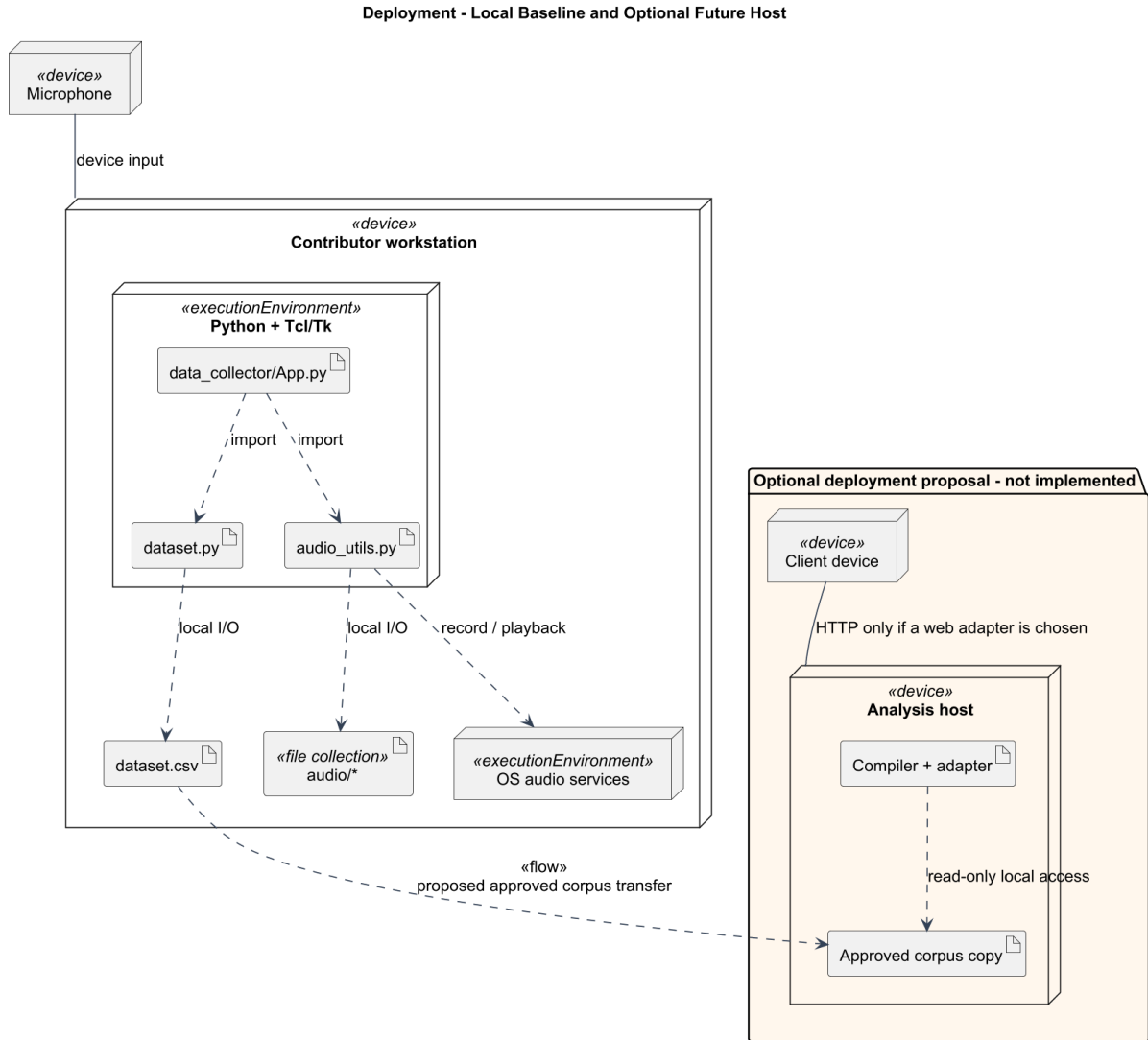


Figure 2: Single-workstation deployment and optional future analysis host

3 Static Structure and Data Design

3.1 Application classes and supporting modules

Trace: FR-01–FR-14, FR-17/FR-18; NFR-02–NFR-05.

Source evidence: `classApp(ctk.CTk)`, `classRecorder`, CSV field definitions.

Figure 3 includes both application-defined classes: `App` and `audio_utils.Recorder`. Selected external classes are `customtkinter.CTk` and `sounddevice.InputStream`; individual widget classes are omitted to keep the domain relations visible.

- **App** inherits the window behavior of `CTk`. Its pending filename belongs to the unsaved draft; the selected ID/audio filename refer to a browse selection. Save, reset, recording, attachment and maintenance methods delegate to helpers rather than embedding CSV serialization in widgets.

- **Recorder** buffers sample arrays and tracks recording state. Start creates a stream; Stop closes it, clears its reference and returns an array or no data. The App, not the helper module, owns the recorder instance.
- **Dataset module** exposes static-looking UML operations for actual module-level Python functions. It is not an instantiated repository class.
- **Audio module** exposes save/copy/play functions and capability constants. Its functions do not own an App's recorder.
- **DatasetEntry** is the CSV/Python-dictionary schema. DatasetFile and AudioFile are logical storage entities, not invented Python classes.

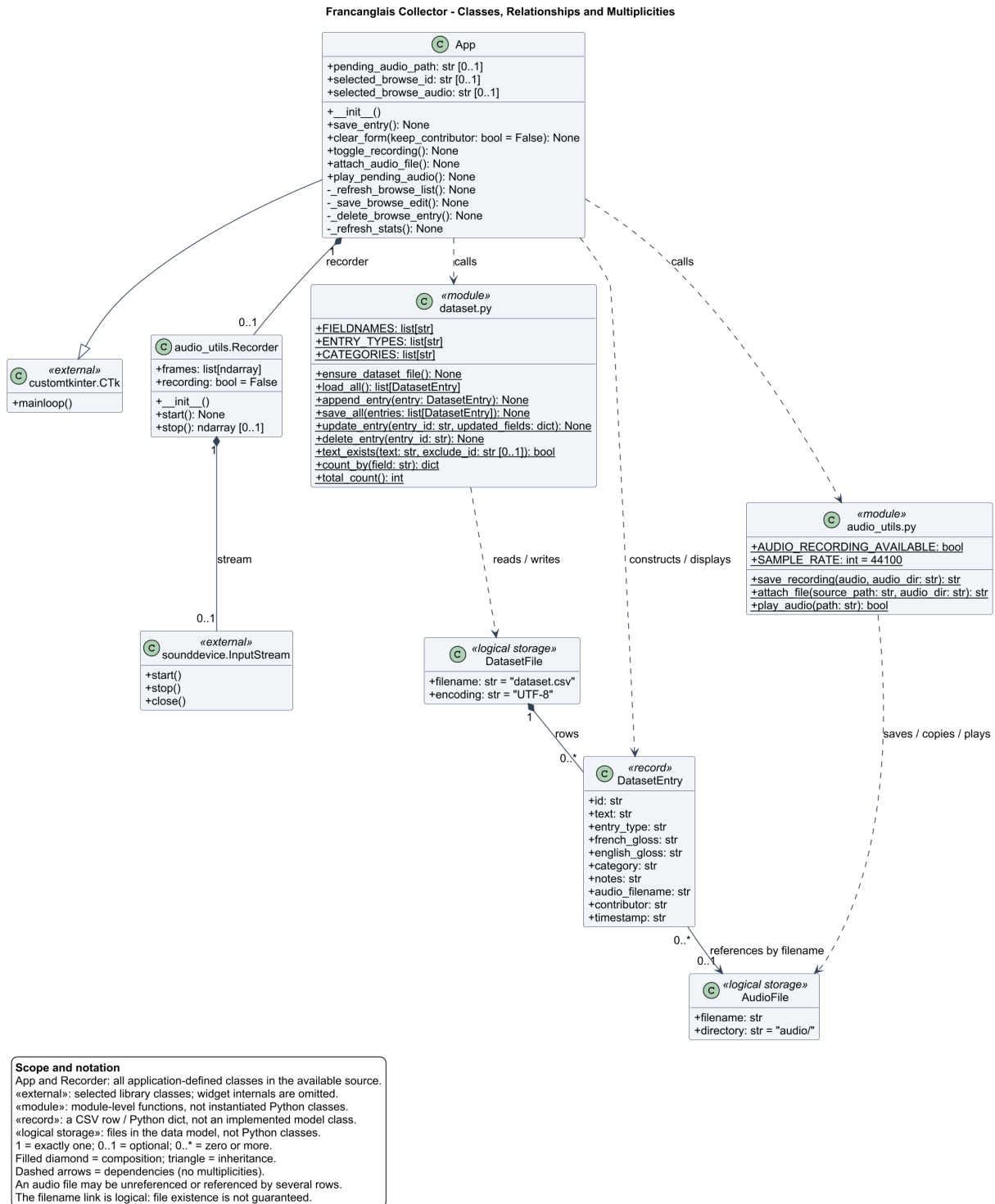


Figure 3: Application classes, module dependencies and cardinalities

Association	Multiplicity and rationale
App–Recorder	One app owns zero or one recorder, depending on optional imports. Every recorder constructed by the collector belongs to that app.
Recorder–InputStream	A recorder owns zero or one live stream. Its reference is absent before Start and after Stop.

Association	Multiplicity and rationale
DatasetFile–	The local CSV contains zero or more row records; each stored row belongs to that file in this model.
DatasetEntry	
DatasetEntry–AudioFile	
	One row may reference zero or one filename. A file may be referenced by several rows or by none; filename uniqueness is not enforced across rows.

The audio association is not composition: removing a row does not remove its file, and clearing a draft can leave staged media behind. A filename link also does not prove that its file exists. Dependency arrows have no cardinality, and inheritance is not an ownership relation.

3.2 Persistence and invariants

The canonical row has ten fields: ID, text, type, French gloss, English gloss, category, notes, audio filename, contributor and timestamp. Source constants define the three entry types and ten categories; the GUI controls their selection but the helper module does not validate arbitrary external CSV edits. IDs are ten hexadecimal characters derived from a UUID with no collision check.

`append_entry` writes one row. `update_entry` loads every row, updates the first matching ID, then rewrites all rows. `delete_entry` filters all matching IDs and rewrites retained rows. `save_all` is not transactional or failure-atomic. Backups, safe replacement and explicit mutation errors are target work in PB-11, not baseline invariants silently added to the diagrams.

4 User Goals and Control Flow

4.1 Use-case diagram

Trace: FR-02–FR-14, FR-18; US-02–US-10.

Source evidence: Collect, Browse & Edit and Stats actions in `App.py`.

Figure 4 models actor goals within the implemented collector. A contributor collects examples, finds them, plays audio and inspects statistics. A maintainer specializes the contributor role and also edits and deletes. The application does not authenticate these roles; the generalization describes responsibilities, not permissions.

Relationship reading: collection and editing include non-empty-text validation; deletion includes confirmation for a selected row. These arrows point to required sub-behavior. Recording and attachment extend collection at the before-save extension point only when chosen. Extension arrows point from optional behavior to the base use case, never in the reverse direction. The text-only path remains complete without either extension.

Storage files are not actors. Internal persistence is not presented as a standalone user goal. Proposed compiler-user stories remain in the SRS rather than appearing inside an already implemented collector boundary.



Figure 4: Collector actors, mandatory sub-behavior and optional audio extensions

4.2 Entry-collection activity diagram

Trace: FR-02–FR-10, FR-17; US-02, US-03, US-07, US-09.

Source evidence: `save_entry`, `attach_audio_file`, `toggle_recording`.

Figure 5 follows one draft through a save attempt. The contributor supplies text, type, category and optional annotations, then may add audio. Recording and attachment are user choices; having a microphone does not force the recording path. Cancellation leaves the

draft unchanged, and a no-frame capture does not create a new WAV.

Validation happens when Save is requested, after optional media staging. Non-empty trimmed text generates a row ID/time, appends the row, refreshes recent/count displays and clears the form while preserving contributor. Empty text leaves the draft available for correction and displays a warning. The final node ends this attempt only. The app can accept another attempt.

The advisory duplicate warning is not a blocking decision. File/device exceptions are outside this normal-flow activity; recording a file before an invalid save can leave unreferenced media, which is not automatically deleted.

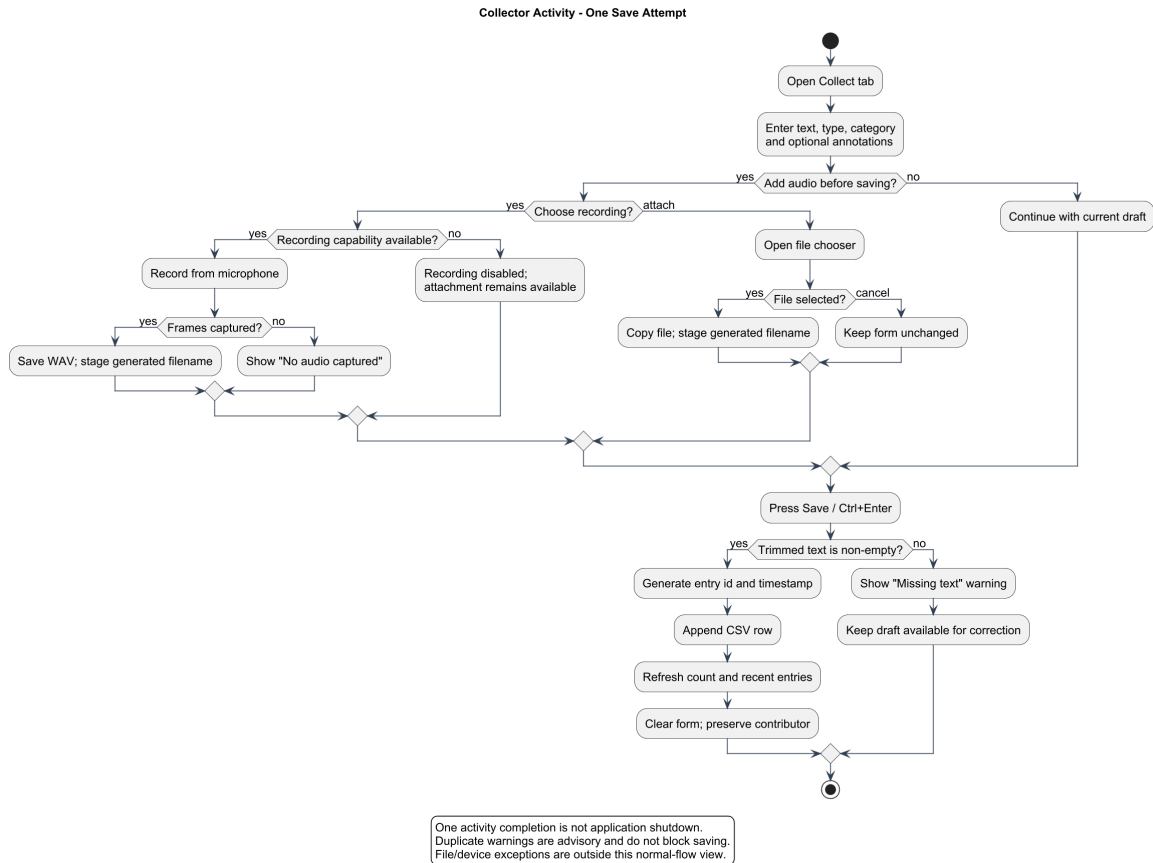


Figure 5: Collection activity with optional media and save-time validation

4.3 Entry workflow state diagram

Trace: FR-02, FR-12, FR-17/FR-18; US-02, US-05, US-06.

Source evidence: draft fields, selected browse ID, edit and confirmation handlers.

Figure 6 is a conceptual UI/persistence state model, not a Python enum or state-machine library. Draft means an unsaved form; Persisted means a row exists; Selected means its fields are loaded for editing. Audio staging is a Draft self-transition, not proof of a saved dataset entry.

Save leads to a choice: empty text returns to Draft with a warning; non-empty text reaches Persisted through an append. A selected edit similarly branches on text validation, with both outcomes returning to Selected and only a valid one updating storage. Deletion waits for a choice: No preserves selection; Yes removes the row, clears selected state and ends that row's workflow. Mutually exclusive guards make each decision explicit.

An edit does not create a new row ID or timestamp. Deleted does not mean that audio was destroyed. The model assumes successful I/O and deliberately omits unimplemented undo, concurrency and recovery states.

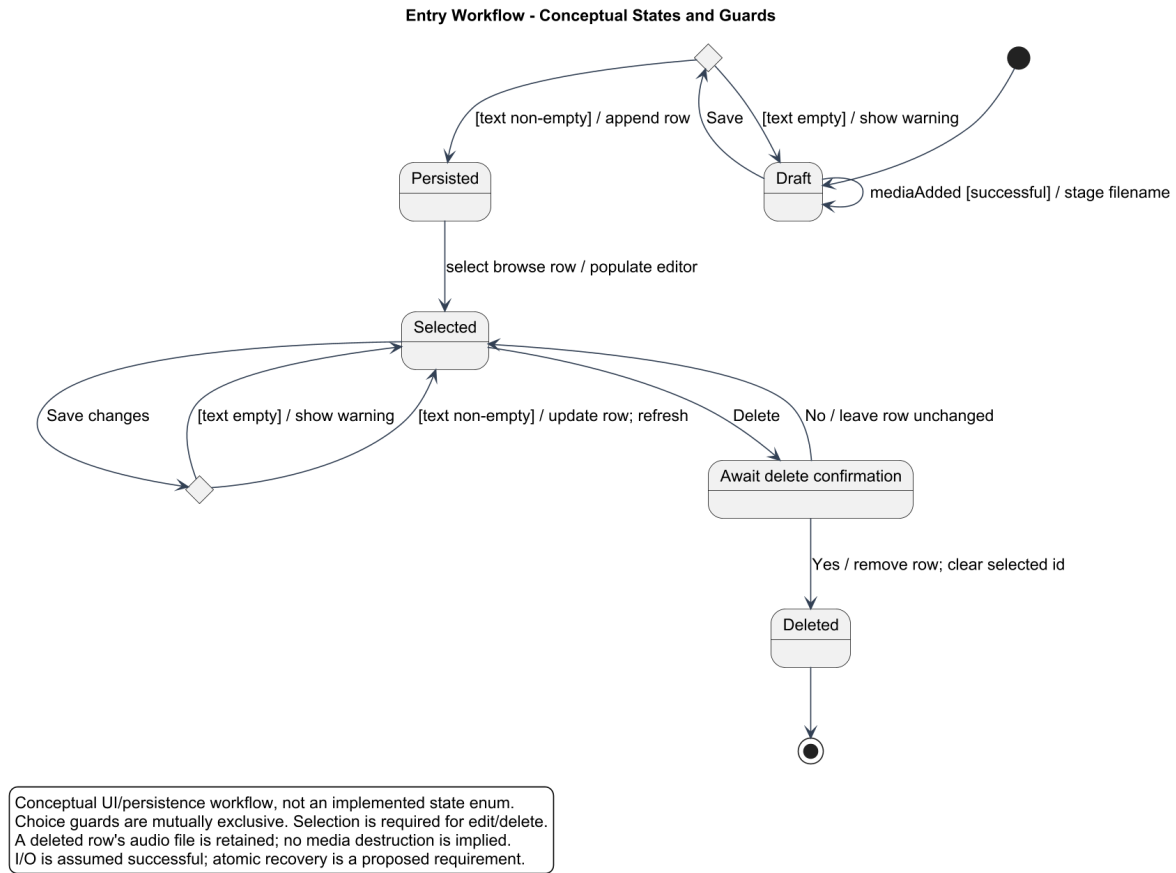


Figure 6: Conceptual draft, selection, edit and deletion states

5 Sequence Diagrams and Interaction Explanations

5.1 Common sequence conventions

All seven sequences show source-grounded collector interactions. The user initiates a handler; App activates; synchronous helper calls activate their receivers; replies close those executions. A module self-call adds a nested activation. A waiting App remains active until its handler returns. The actor's reply denotes completion of the visible interaction, not an additional network response.

Successful disk operations are assumed unless a failure branch is explicitly drawn. A note may summarize repeated refresh/widget/library work; it does not claim those operations disappeared from the implementation. Unavailable compiler source is not replaced by fictional compiler sequence diagrams.

5.2 Collect and save with optional attachment

Trace: FR-01–FR-06, FR-09/FR-10, FR-17; US-01/US-02/US-07.

Source evidence: attach_audio_file, save_entry, append_entry.

Precondition: the collector has initialized storage and the contributor is preparing a draft. Figure 7 begins with an optional selected-file attachment; cancellation is outside that successful optional fragment.

1. App asks the audio module to copy the selected source into the managed directory, receives a generated filename and stages it. No CSV row exists yet.
2. Save strips the text. The empty branch displays a warning and never calls persistence. The valid branch builds a dictionary with metadata and the pending filename, or an empty audio field.
3. The dataset module ensures storage, appends one CSV row and returns. Its initialization call has a nested activation inside the append operation.
4. App retrieves total rows and recent entries, then clears the form with contributor retention. The final reply reflects warning or saved state.

Postcondition and limits: a valid row is stored if I/O succeeds. An advisory duplicate may still be saved. Reset clears the pending reference but does not delete a file; filesystem errors are not converted into success.

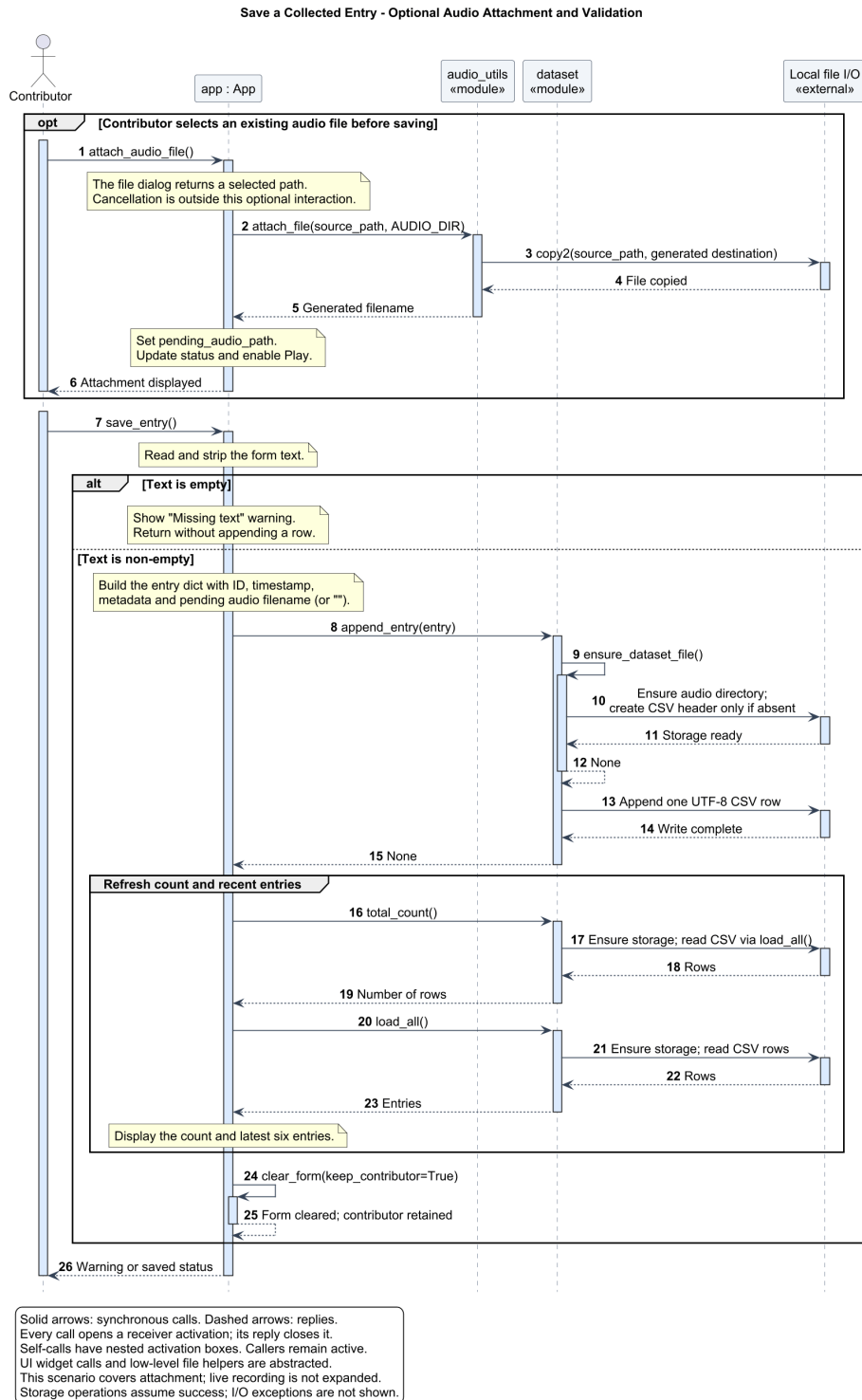


Figure 7: Save interaction with optional attachment and validation branches

5.3 Search and select an entry

Trace: FR-11; US-04.

Source evidence: `_refresh_browse_list`, `_on_browse_select`.

Figure 8 separates filtering from selection.

1. A search key release or Refresh invokes the handler. It strips/lowercases the query, removes old tree rows and resets the ID-to-entry cache.

2. One dataset load returns CSV dictionaries. The loop tests a combined string of each row's text, category and contributor; an empty query accepts every row.
3. Matches are added both to the cache and to the visible tree. A non-match is skipped; zero matches is a valid empty result, not a read failure.
4. An optional row selection reads the cache, sets selected ID/audio and populates edit controls. It performs no new CSV read.

Postcondition and limits: selection refers to a displayed cached entry. An absent selection/cache entry returns without a new mutation. External concurrent edits can make that cache stale; there is no locking.

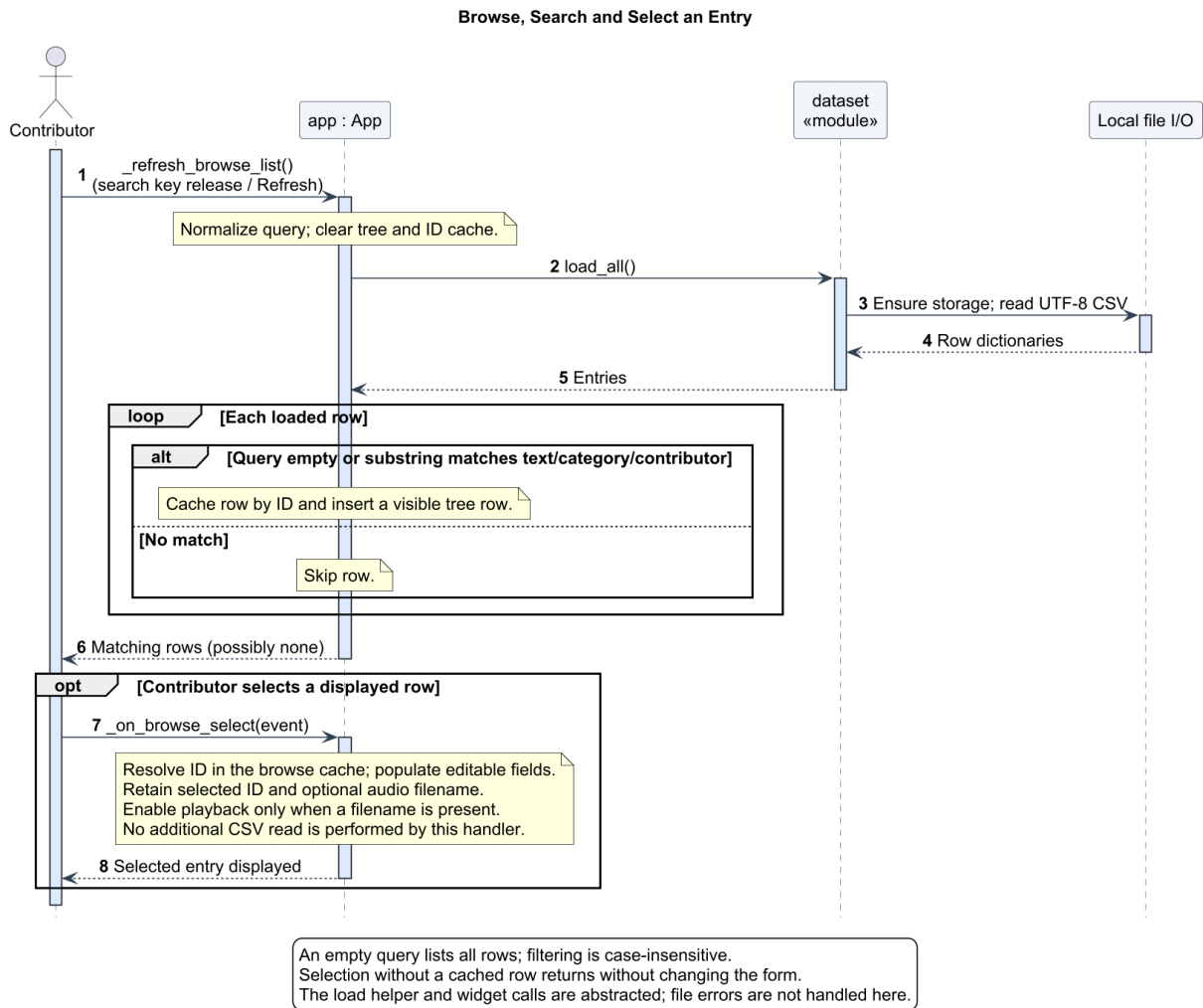


Figure 8: Searching CSV entries and selecting a cached result

5.4 Edit an existing entry

Trace: FR-06, FR-12, FR-18; US-05.

Source evidence: `_save_browse_edit`, `update_entry`, `save_all`.

Figure 9 shows three guarded cases: no selected ID, empty replacement text, and a valid selected edit. The first two show messages without writing. On the valid path:

1. App builds updates only for text, type, category, glosses and notes.

2. The dataset helper calls `load_all` on itself, updates the first matching ID, then calls `save_all`. Both self-calls have nested boxes.
3. The writer rewrites the header and all rows. After its return, App shows the status and refreshes browse/count/recent displays.

Postcondition and limits: identity, original timestamp, contributor and audio link are preserved. If an ID has disappeared externally, the helper does not report a missing-ID error; it rewrites unchanged rows. The current direct rewrite can fail partway through, which FR-19/NFR-11 propose to address.

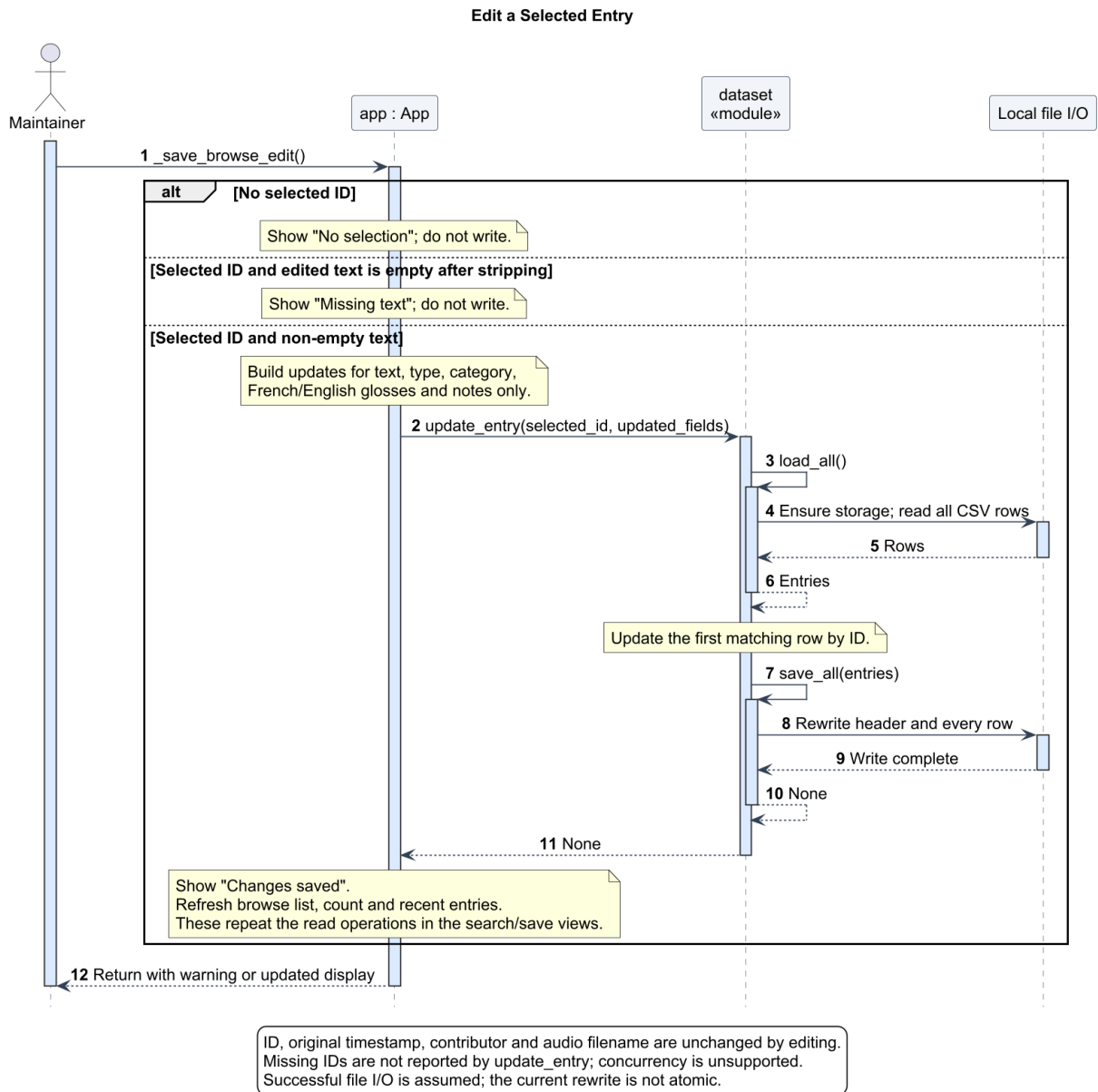


Figure 9: Selected-entry validation and read-modify-write editing

5.5 Confirm or cancel deletion

Trace: FR-12, FR-18; US-06.

Source evidence: `_delete_browse_entry`, `delete_entry`, confirmation dialog.

Figure 10 makes cancellation a real alternative, not a note following an unconditional deletion. With no selected ID, App only displays information. Otherwise the modal `askyesno` call stays active while awaiting the maintainer's choice.

A No reply returns without persistence or selected-state changes. Yes calls the dataset helper, which loads rows, removes all rows matching the selected ID and writes the retained rows. App then clears its selected ID and audio reference and refreshes the displays.

Postcondition and limits: the confirmed row is no longer in the CSV if I/O succeeds; the audio file remains. There is no undo operation, deletion audit log or atomic replacement in this baseline. Low-level file helpers and repeated UI refresh reads are condensed in the figure.

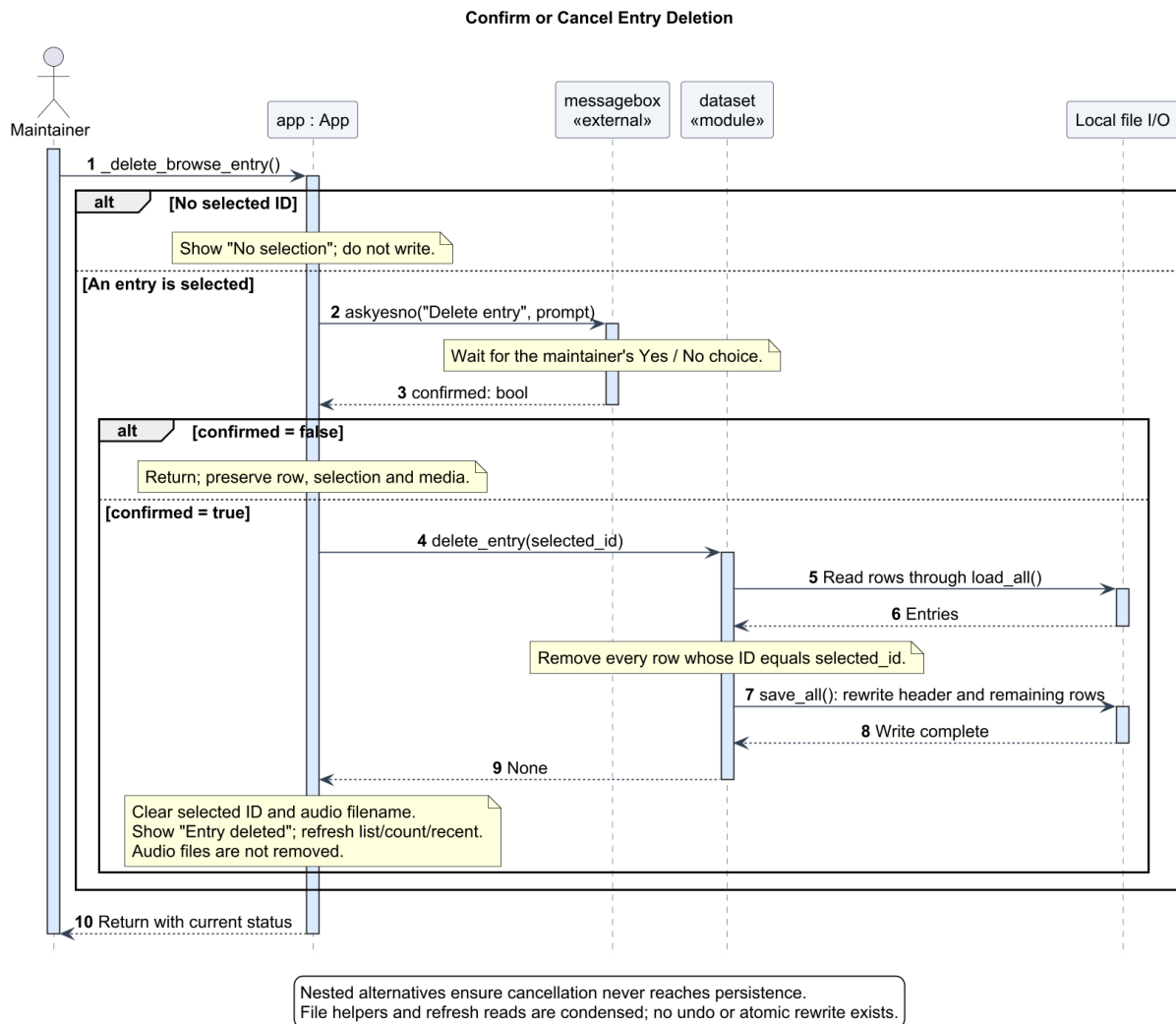


Figure 10: Selection guard, confirmation and conditional deletion

5.6 Record and stage microphone audio

Trace: FR-08/FR-10, NFR-05/NFR-11; US-09.

Source evidence: `toggle_recording`, `Recorder.start`, `Recorder.stop`, `save_recording`.

Figure 11 assumes optional imports succeeded, a recorder exists and the device opens. Record and Stop are separate user interactions.

1. Record starts a Recorder execution that clears prior frames, sets the recording flag and creates an InputStream with the callback, one channel and 44,100 Hz sampling. The stream lifeline begins at its creation message.
2. Starting the stream returns control to the UI, which shows recording status. Between user actions, callbacks copy frames while the flag is true. Those asynchronous callbacks are summarized, not misdrawn as a long blocking call.
3. Stop clears the flag, stops/closes the stream and releases its reference. The helper concatenates buffered frames or returns no data.
4. No frames yields an empty-capture message. Otherwise the audio module writes a generated WAV and returns the filename to stage in the draft.

Postcondition and limits: a successful WAV still needs a separate form Save to become linked to a row. Import failure disables recording; device-open/stop/write failures are not caught by these handlers. Error hardening is target PB-09, and no unlimited-recording guarantee is made.

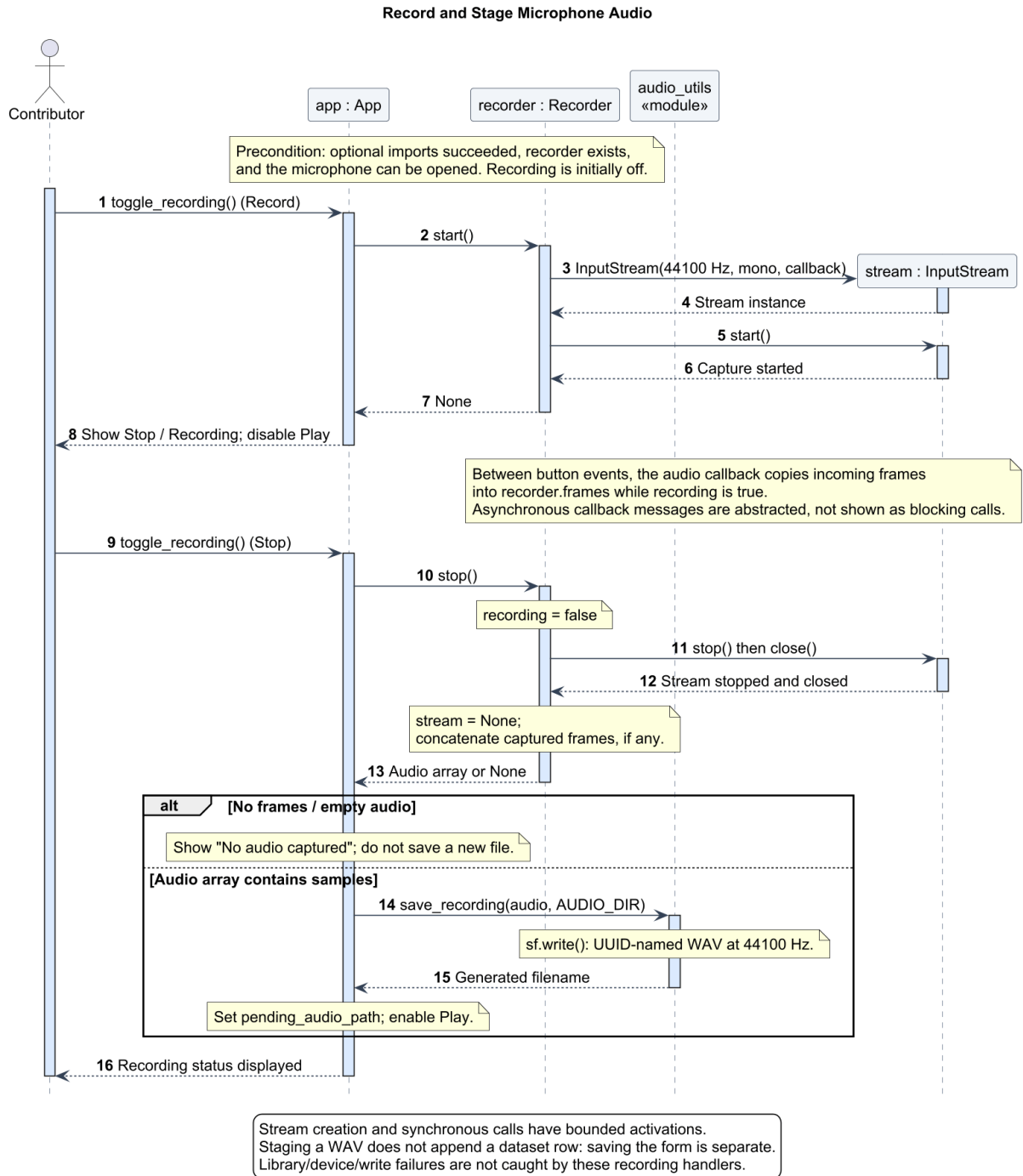


Figure 11: Recorder start, stream lifetime, stop and staged WAV creation

5.7 Play audio and fall back to the operating system

Trace: FR-13, NFR-03/NFR-05; US-08.

Source evidence: play_pending_audio, _play_selected_audio, play_audio.

Figure 12 starts with a pending or selected filename. Without such a filename the GUI handler returns before this sequence.

1. App resolves the managed path and asks the audio module to play it. Missing files immediately produce a false result without trying a player.

2. If optional libraries are available, soundfile reads the media and sounddevice receives a playback request. Successful invocation returns true.
3. If libraries are unavailable or that attempt raises, the helper tries the platform default-player launch. An accepted launch returns true; an exception returns false.
4. App shows “Playback failed” only for false; playback does not modify rows.

Result semantics: true proves only that a library or OS accepted a request. It does not prove audible output, complete playback or remote-process success after launch. This distinction prevents a false end-to-end quality claim.

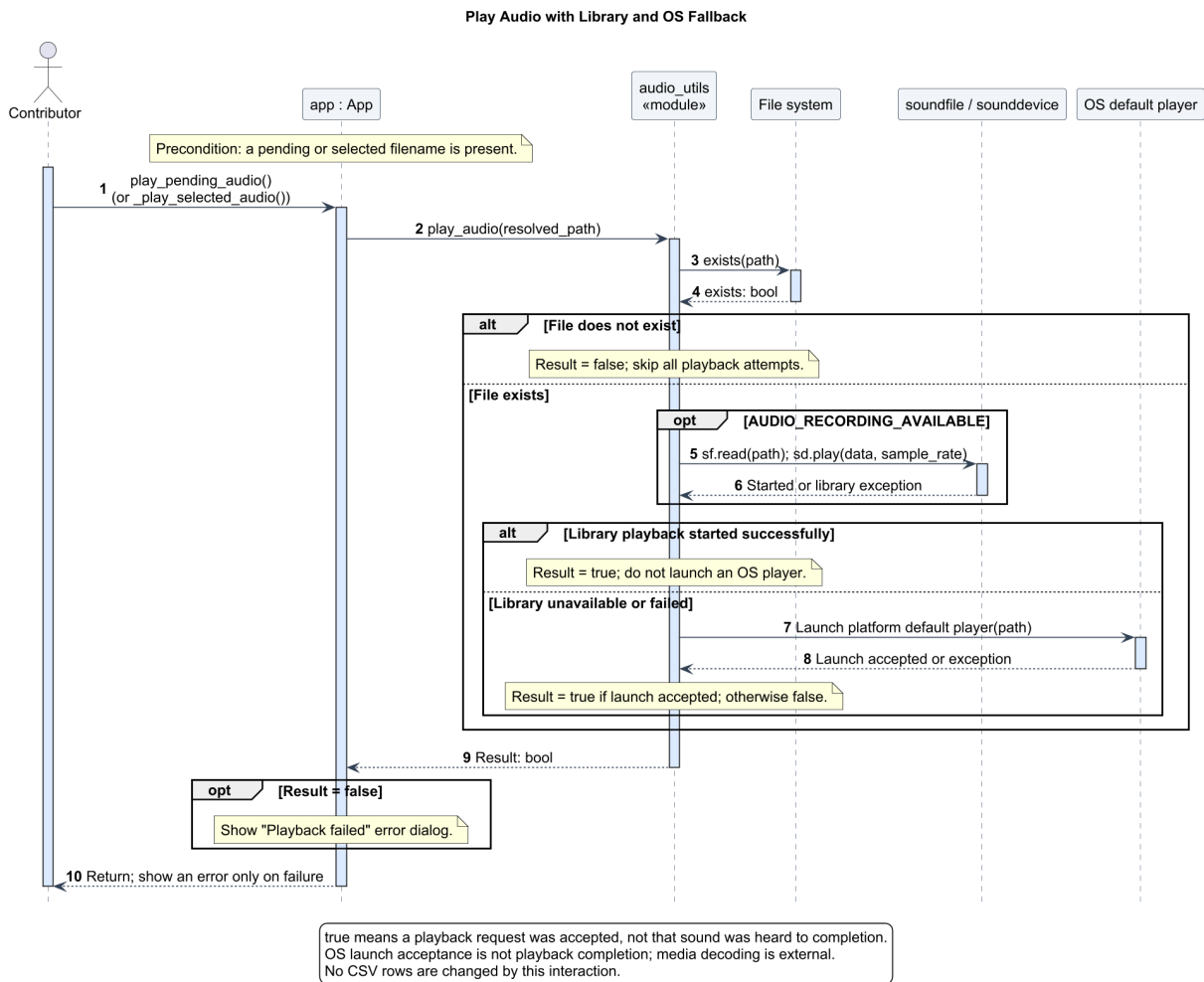


Figure 12: Playback existence check, library attempt and OS fallback

5.8 Refresh statistics

Trace: FR-14, NFR-10; US-10.

Source evidence: `_refresh_stats`, `count_by`, `_render_bar_group`.

Figure 13 uses the explicit Refresh button, a verified entry point. It does not rely on the tab-change event being delivered by a particular widget version.

1. App removes old bars but keeps headings, then obtains the total row count.

2. In order, it requests counts for entry type and category. Each `count_by` rereads the dataset and uses “(none)” for missing/empty values.
3. A nested render call shows “No data yet” for empty counts, otherwise sorts counts descending and draws a bar proportional to count/total with a zero-total guard.

Postcondition and limits: the display represents the reads just performed. There are three CSV reads, not one atomic snapshot; concurrent writers could create inconsistent totals. No new rows or files are created apart from storage initialization if missing.

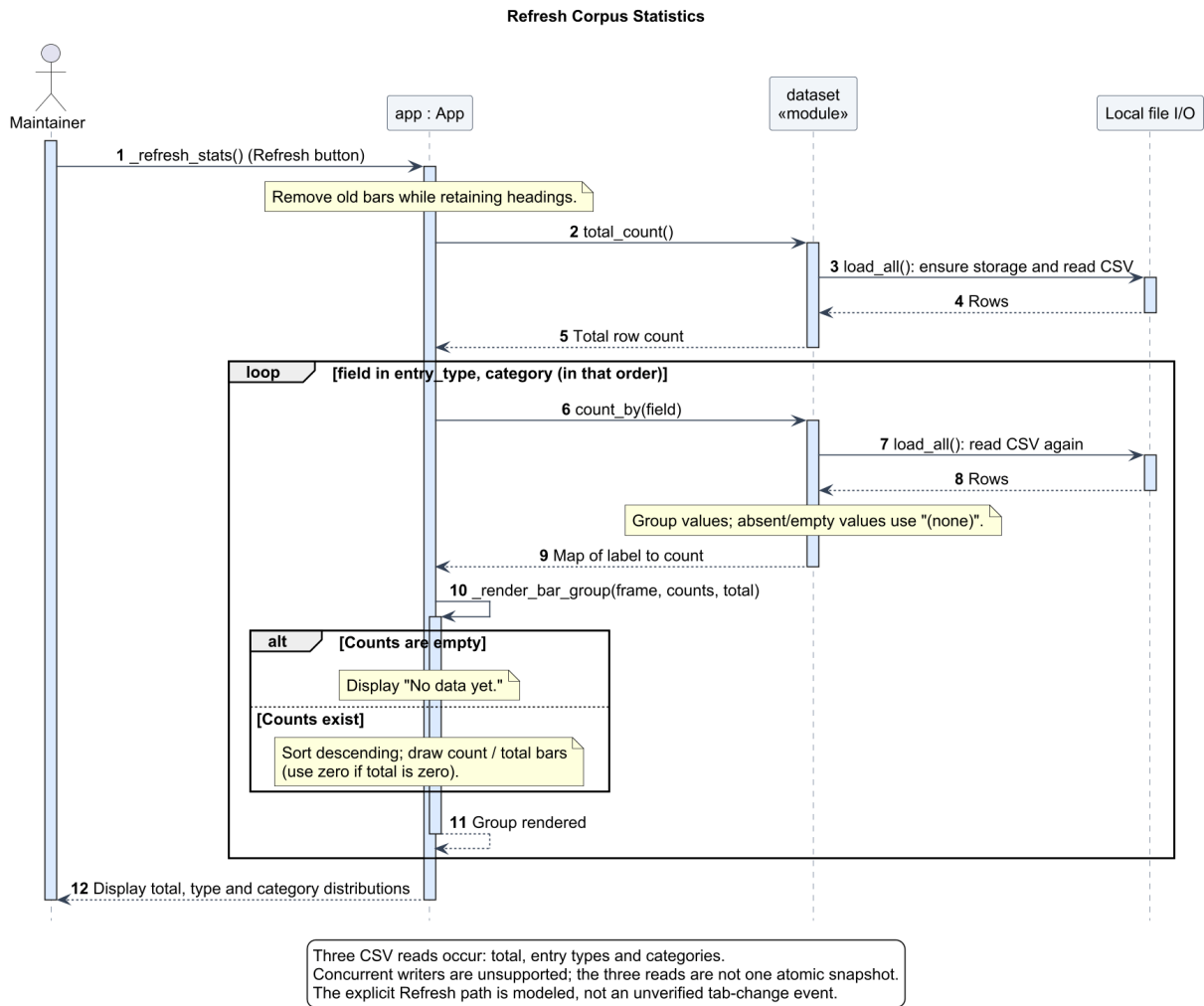


Figure 13: Statistics refresh, repeated grouping and safe empty-data rendering

6 Interfaces, Errors and Design Limitations

Interface	Actual contract or explicitly proposed target
<code>dataset.append_entry(entry)</code>	Append a canonical dictionary as one row; I/O errors propagate.
<code>dataset.update_entry(id,fields)</code>	Update the first matching row and rewrite all; no missing-ID result.

Interface	Actual contract or explicitly proposed target
<code>dataset.delete_entry(id)</code>	Filter matching IDs and rewrite retained rows; no audio cleanup.
<code>dataset.count_by(field)</code>	Return label/count pairs from a new CSV load, using “(none)” for empty values.
<code>audio_utils.attach_file(source, dir)</code>	Copy with generated filename and original extension; return filename, not a validated transcription.
<code>Recorder.stop()</code>	Clear recording, close/release stream, return concatenated frames or None.
<code>audio_utils.play_audio(path)</code>	Return accepted-start/launch boolean; no guarantee of completed playback.
Proposed analysis adapter	Accept text/configuration and return lexical/sentence evidence without mutating corpus; no current route is asserted.

6.1 Failure boundaries

The diagrams show current guards and observable behavior, not a redesigned error layer. GUI save/edit reject empty text and deletion asks confirmation. Playback catches library/OS failures to attempt fallback or return false. CSV writes, attachment copies and recording operations do not consistently surface recoverable errors; target FR-19/NFR-11 require that hardening.

Read-modify-write has neither locking nor atomic replacement. Cached IDs can be stale, duplicate IDs can affect more than one deletion, and orphan audio can remain after cancelled drafts or deleted rows. These are limitations, not features to hide behind optimistic success paths.

6.2 Privacy and operational boundaries

No secret environment values or raw personal recordings belong in diagrams or PDFs. Repository ignore rules reduce accidental inclusion but do not prove past history is free of data. Consent, access restrictions and controlled backup copies are team procedures with acceptance evidence in PB-11/PB-15. The proposed host requires explicit, approved corpus transfer.

7 Verification, Traceability and Change Impact

Diagram	Stories	What its verification must establish
Component, Fig. 1	US-01–US-14	Local dependency directions and visibly proposed analysis boundary.
Deployment, Fig. 2	US-07–US-15	Correct workstation/device artifacts; no imaginary shared filesystem.

Diagram	Stories	What its verification must establish
Class, Fig. 3	US-01–US-10	Real classes/methods, justified multiplicities and labelled logical entities.
Use case, Fig. 4	US-02–US-10	Actor goals; include/extend/generalization direction.
Activity, Fig. 5	US-02/US-07/US-09	Optional media choice and validation at Save, not earlier.
State, Fig. 6	US-02/US-05/US-06	Exclusive guards and retention of selection on edit/cancel.
Save, Fig. 7	US-01/US-02/US-07	Conditional persistence and balanced nested calls.
Search, Fig. 8	US-04	Query filter, cache and no extra selection read.
Edit, Fig. 9	US-05	Allowed-field update and preservation of metadata.
Delete, Fig. 10	US-06	No write after cancellation and no media deletion.
Record, Fig. 11	US-09	Stream creation/start/stop and no WAV without frames.
Playback, Fig. 12	US-08	Missing-path and fallback branches with honest return semantics.
Stats, Fig. 13	US-10	Three reads, grouping order and safe empty rendering.

Documentation checks render each PlantUML source locally, check paired call/reply activations, compile both LaTeX documents and inspect the PDFs. Acceptance testing of the application is a separate backlog obligation: use temporary fixtures, mock audio interfaces, inject failures and record results rather than treating diagram compilation as proof of runtime behavior.

When a handler or helper contract changes, update its sequence, related activity/state/class view, SRS acceptance criteria and affected backlog item. Keep the `theme.iuml` include beside the PlantUML sources so rendering uses one consistent style. Group names/matricules remain placeholders until the team supplies them.

8 Proposed Evolution

PB-11 adds safe persistence, regressions and recovery procedures before analysis work relies on corpus quality. PB-13/PB-14 propose a read-only lexer, classifier and declared-rule sentence-analysis increment, not a production language standard. PB-16 retains unknown-word prediction as deferred research. If a web adapter or database is later approved, revise deployment, interfaces, privacy and concurrency requirements before treating it as implemented design.